

Concepts Explained Through the Food Delivery Platform

Theory + real examples from the Java 17 / Spring Boot 3 project you built
(Zomato/Swiggy-style microservices)

Topics: Monolith vs Microservices, Benefits, Challenges, Service Boundaries, Single Responsibility, Database per Service, Inter-Service Communication

Project: food-delivery-platform — 11 microservices + API Gateway + Eureka + MySQL

Table of Contents

1. 1. What Are Microservices?
2. 2. Monolith vs Microservices
3. 3. Benefits of Microservices
4. 4. Challenges of Microservices
5. 5. Identifying Service Boundaries (Business Domains)
6. 6. Single Responsibility Principle in Service Design
7. 7. Database per Service
8. 8. Communication Between Services
9. 9. When NOT to Use Microservices
10. 10. Quick Reference — Our Project Map

1. What Are Microservices?

Microservices is an architectural style where an application is built as a collection of **small, independent services**. Each service:

- Runs in its own process
- Owns a focused business capability
- Can be developed, deployed, and scaled independently
- Communicates with others over the network (usually HTTP/REST)

Why are they used?

- Large systems become too big to maintain as one codebase (monolith)
- Different parts of the business change at different speeds
- Teams need to deploy without waiting for the entire application
- Scale only the busy parts (e.g. order service during peak hours)

***In our project:** We split the food delivery app into 11 services: Auth, Customer, Restaurant, Menu, Order, Delivery Partner, Payment, Notification, plus Discovery, Config, and API Gateway. Each is a separate Spring Boot application with its own JAR and port.*

2. Monolith vs Microservices

Aspect	Monolith (single app)	Microservices (our project)
Structure	One codebase, one deployment	Many repos/modules, many deployments
Database	Usually one shared database	Database per service (8 MySQL DBs)
Scaling	Scale entire application	Scale only order-service if orders spike
Technology	One stack for everything	Can mix stacks per service (we use Java 17)
Failure	Bug can crash whole system	Payment down; customers may still browse menus
Complexity	Simpler to start	Needs Gateway, Eureka, Config, Feign

MONOLITH



MICROSERVICES (Our Project)



***In our project:** If this were a monolith, all controllers would live in one project with one MySQL schema. Instead, OrderController is only in order-service, and Payment logic is only in payment-service.*

3. Benefits of Microservices

3.1 Scalability

You can run **more instances** of only the services under heavy load. Eureka registers multiple instances; the API Gateway load-balances requests.

***In our project:** During lunch rush, you could run 5 instances of order-service and 1 instance of notification-service. Eureka + Gateway (lb://order-service) distribute traffic automatically.*

3.2 Independent Deployment

Change payment logic and deploy **only payment-service** without redeploying customer, restaurant, or menu services.

***In our project:** Docker: `docker compose build payment-service` && `docker compose up -d payment-service`. Other containers keep running. No full-system downtime.*

3.3 Fault Isolation

If one service fails, others may continue working (if designed with fallbacks). A crash in notification-service should not take down customer registration.

***In our project:** If notification-service is down, order placement might fail at the Feign call unless you add circuit breakers (Resilience4j — future improvement). Today, order-service depends on notification for success — a real trade-off to study.*

3.4 Team Autonomy

Team A owns Restaurant + Menu; Team B owns Order + Delivery. Each team chooses release cycles, coding standards within shared rules, and owns their database.

***In our project:** Folder structure mirrors ownership: `restaurant-service/` and `menu-service/` are separate Maven modules. Teams commit independently to the monorepo or separate repos.*

4. Challenges of Microservices

4.1 Complexity

You manage many services, configs, deployments, and versions instead of one application.

***In our project:** We need: Eureka (8761), Config Server (8888), API Gateway (8080), MySQL, Docker Compose, 8 business services, JWT, Feign clients. A monolith would need only one `application.yml`.*

4.2 Network Latency

Every call between services is an HTTP request over the network — slower than an in-process method call inside a monolith.

***In our project:** `POST /orders` triggers 4+ Feign calls (customer, restaurant, payment, notification). Each adds milliseconds. Acceptable for most apps; problematic for ultra-low latency.*

4.3 Data Consistency

No single database transaction across services. You cannot easily do ACID across order_db and payment_db. Patterns: Saga, eventual consistency, outbox.

***In our project:** Order is saved, then payment is processed. If payment fails after order save, you need compensation logic (cancel order) — we simulate payment as always SUCCESS. Production needs Saga or two-phase patterns.*

4.4 Monitoring

You must trace requests across services (correlation IDs, distributed tracing). Logs are scattered across 11 containers.

***In our project:** docker compose logs order-service shows order flow; payment logs are separate. Tools like Zipkin, ELK, or Grafana Loki are added in production.*

5. Identifying Service Boundaries Using Business Domains

Split services by **business capability** (Domain-Driven Design), not by technical layers. Ask: "What does the business do?" not "Where should controllers go?"

Business Domain	Service	Why separate?
Identity & login	auth-service	Security is a distinct concern
Customer profiles	customer-service	User data lifecycle
Restaurants	restaurant-service	Supply side of marketplace
Menus	menu-service	Changes often; tied to restaurant but own data
Orders	order-service	Core workflow; orchestrates others
Delivery	delivery-partner-service	Logistics domain
Payments	payment-service	PCI, refunds, gateways — isolate risk
Notifications	notification-service	Email/SMS/push — cross-cutting

In our project: Wrong boundary: "database-service" or "controller-service" (technical split). Right boundary: "menu-service" owns menu items because the business thinks in menus.

6. Single Responsibility Principle in Service Design

SRP: A service should have only one reason to change. One cohesive business responsibility.

- payment-service changes when payment rules change — not when menu UI changes
- menu-service changes when menu fields change — not when JWT format changes
- auth-service only handles register, login, JWT — not order placement

In our project: Each service has one main package job: `CustomerServiceImpl` only manages customers. `OrderServiceImpl` places orders but does NOT store payment rows — it calls `payment-service`.

7. Database per Service Concept

Each microservice has its **own private database**. No other service may access that database directly. Data is shared only via APIs (REST/Feign).

Service	Database	Tables (examples)
auth-service	auth_db	users
customer-service	customer_db	customers
restaurant-service	restaurant_db	restaurants
menu-service	menu_db	menu_items

Service	Database	Tables (examples)
order-service	order_db	orders
delivery-partner-service	delivery_db	delivery_partners
payment-service	payment_db	payments
notification-service	notification_db	notifications

Config location in our project:

- config-repo/auth-service.yml — jdbc:mysql://.../auth_db
- config-repo/application.yml — shared username/password, JPA ddl-auto: update
- docker-compose.yml — SPRING_DATASOURCE_URL per service (Docker hostname mysql)
- docker/mysql/init-databases.sql — creates all 8 databases

In our project: Access from your PC (Docker): localhost:3307, user root, password root. Services inside Docker use mysql:3306. You cannot JOIN customers table from order_db.

8. Communication Between Services (High-Level)

Types of communication:

Type	Description	In our project
Synchronous	Caller waits for response (HTTP)	OpenFeign REST calls
Asynchronous	Message queue; caller does not wait	Not used (could add Kafka)
Client → Gateway	Single entry point	API Gateway :8080
Service discovery	Find service instances	Eureka :8761

```
Client --> API Gateway (8080) --> customer-service
                                   --> order-service
                                       |
                                       | Feign (HTTP + DTO only)
                                       v
+-----+-----+-----+-----+
|Customer|Restau- |Payment |Notifi- |
|Service |rant   |Service |cation |
+-----+-----+-----+-----+
```

Rules we follow:

- No entity shared between services
- Feign uses Request/Response DTOs only
- No direct database access across services

Order placement — Feign chain:

- 1. GET customer-service — validate customerId
- 2. GET restaurant-service — validate restaurantId
- 3. SAVE order in order_db
- 4. POST payment-service — process payment
- 5. POST notification-service — ORDER_PLACED, ORDER_ACCEPTED

In our project: Feign interfaces live in order-service/client/: CustomerClient, RestaurantClient, PaymentClient, NotificationClient, DeliveryPartnerClient. @FeignClient(name = "customer-service").

9. When NOT to Use Microservices

- **Small project / startup MVP** — monolith is faster to build and deploy
- **Tiny team (1–3 developers)** — operational overhead of 11 services is too much
- **Simple CRUD app** — no need for distributed complexity
- **Strong ACID across everything** — bank transfer style transactions need one DB
- **No DevOps maturity** — Docker, monitoring, CI/CD not ready
- **Unclear domain boundaries** — you will split wrong and regret it

In our project: A college assignment or food delivery prototype with 2 developers could start as a monolith. We chose microservices to **learn** enterprise patterns — that is a valid educational reason, even if a monolith would ship faster for production v1.

10. Quick Reference — Our Project Map

Concept	Where to see it in project
Microservices	11 modules in pom.xml <modules>
API Gateway	api-gateway/ — routes in application.yml
Service Discovery	discovery-server/ — Eureka UI :8761
Config Server	config-server/ + config-repo/
Database per service	config-repo/*-service.yml + 8 MySQL DBs
Feign communication	order-service/.../client/*.java
DTO boundary	*/dto/*RequestDto.java, *ResponseDto.java
Layered architecture	controller / service / repository per service
JWT Security	auth-service/security/
Swagger testing	http://localhost:8080/swagger-ui.html
Docker run	docker-compose.yml, DOCKER.md

Study path: Read this PDF → Explore Eureka → Test Swagger (SWAGGER-TESTING.md) → Read OrderServiceImpl (Feign) → Connect MySQL on port 3307 and view tables.

Food Delivery Platform — Microservices Learning Guide